

DLC: A Multi-Model, Adaptive-Precision Near-Lossless Compression Engine for High-Frequency Float64 Time-Series Telemetry

Anonymous Author(s)

Anonymous Institution

Abstract. Most modern industrial sensors, IoT gateways, phasor units and scientific instruments emit their readings as IEEE 754 double-precision (float64) values. When those bytes are fed into a general-purpose dictionary compressor like GZIP or ZLIB, the resulting compression ratio is typically close to one. The cause is structural: roughly the bottom 30–40 bits of every mantissa carry hardware thermal noise, so the byte stream contains very few repeating substrings for a Lempel–Ziv dictionary to exploit. We call this the *Entropy Trap*. Strictly lossless float coders such as Gorilla and Chimp move past dictionary matching with XOR-based delta encoding, but they still faithfully encode the noise floor and so their ratios on real sensor traces remain modest. Recent lossless advances, such as Elf’s erasing-based encoding and ALP’s adaptive integer projection, push the lossless frontier further but remain fundamentally bound by the noise floor they must preserve.

This paper describes DLC (Delta-Linear Compression), a near-lossless engine that works in the *mathematical domain* rather than the byte domain. Each block is broken into variance-homogeneous windows by a greedy doubling search; for every window the engine fits six predictive models in parallel (Linear, Quadratic, XOR-Delta, Constant, Sinusoidal, and Predictive-XOR) and picks the one with the smallest Sum-of-Squared-Residuals. The residuals are then quantized at a per-window precision (8–20 bits) and run through six entropy-coding strategies in a tournament. Two implementations share the same `.dlc` binary specification: a NumPy-based Python reference prototype and a C++17 production engine using `std::async`. Across 21 synthetic and real-world float64 datasets, at a strict 16-bit precision keeping the maximum per-sample reconstruction error below 8×10^{-6} , DLC compresses smaller than every baseline (Gorilla, DoD (16-bit), RLE, GZIP, and ZLIB) on 19 of 21 datasets (90.5%), reaching $2355.7\times$ on a 1-million-sample linear ramp and $102.0\times$ on a recovered signal.

Keywords: time-series compression · floating-point compression · IoT telemetry · predictive coding · residual quantization · adaptive precision · variance-guided segmentation · near-lossless

1 Introduction

Telemetry volumes have grown faster than storage budgets for several years now. By IDC’s reckoning, connected sensors and industrial IoT will be producing well above 180 zettabytes a year by the end of this decade, and a large share of that data lands on disk as 8-byte IEEE 754 doubles [1]. The bandwidth bill, however, usually lands at the edge gateway, where compression has to happen in tight latency and power windows. The unfortunate twist is that the standard tools for the job, namely GZIP and ZLIB, contribute almost nothing on this kind of data.

The reason is rooted in the layout of a float64 value: one sign bit, eleven exponent bits, and a fifty-two-bit mantissa [2]. For a continuous physical signal, successive samples agree on most of the high-order mantissa bits, since the underlying value moves only a little between ticks. The bottom 30 to 40 bits are occupied by hardware thermal noise plus the quantization residue of the ADC, and they look statistically random from one sample to the next. A Lempel–Ziv dictionary [3] scanning the resulting byte stream therefore finds essentially no repeating substrings to exploit. We call this the *Entropy Trap*.

The literature has split into two camps. The first works in the bit domain and refuses to lose any information. Gorilla [4] and Chimp [5] are the canonical examples; more recently, Elf [24] introduced an erasing-based approach that zeroes trailing mantissa bits while proving recoverability, and ALP [25] adaptively encodes pseudo-decimal floats as integers. All achieve strong lossless ratios but remain fundamentally bounded by the noise floor they must preserve. The second camp accepts a bounded loss in exchange for stronger compression: SZ [6,7] and ZFP [8] both predict each sample, take the residual, and quantize it to a user-supplied tolerance.

Before turning to what we built, we state our five research objectives:

RO1: Bypass the Entropy Trap by moving compression into a model-fit / residual-encode pipeline. **RO2:** Hold a strict per-sample error bound of $\pm 8 \times 10^{-6}$. **RO3:** Adapt window size, predictor, quantization width, and entropy strategy *per window*. **RO4:** Maintain a Python reference and C++17 production engine agreeing on the `.dlc` binary. **RO5:** Verify continuously with a 105-test suite (76 Python + 29 C++) running on every commit.

DLC (Delta-Linear Compression) is our attempt to meet those five objectives. The main contributions are:

1. Six predictive models, picked per window by SSR, with preference rules favouring simpler or lossless variants.
2. Variance-guided adaptive windowing via greedy doubling with stagnation guard and binary-search refinement.
3. Adaptive precision (8–20 bits) stored per window, with a derived lower bound $p_{\text{floor}} = 16$.
4. Six entropy strategies trial-encoded per window; the smallest wins.
5. A whole-block DoD fast path that makes DLC a functional superset of standalone DoD encoders.

6. A cross-language binary contract enforced by automated CI testing.
7. True multi-core scaling via `std::async` with deterministic output.

We tested the engine on 21 float64 datasets covering synthetic edge cases, real-world telemetry from the Numenta Anomaly Benchmark [11] and Jena Climate station, and round-trip recovery signals. DLC writes a smaller file than every baseline on 19 of 21 datasets. A 105-test verification suite runs on every commit.

The rest of the paper is laid out as follows. Section 2 reviews the related literature, including recent advances in lossless float compression and edge IoT compression. Section 3 walks through the five-stage pipeline with numbered equations and Algorithm 1. Section 4 covers the dual-language implementation. Section 5 presents a quantitative novelty comparison. Section 6 reports the experimental evaluation including compression ratios, error bounds, throughput, MAE/RMSE metrics, and computational complexity analysis. Section 7 discusses limitations and edge deployment considerations. Section 8 concludes.

2 Related Work and Literature Survey

2.1 Lossless XOR/Delta-Based Float Compression

Lindstrom and Isenbourg’s FPC [12] introduced context-based prediction followed by XOR encoding for float64. Burtscher and Ratanaworabhan extended FPC to higher throughput on scientific data [13]. Pelkonen et al.’s Gorilla [4] adopted XOR encoding for values and became the de facto standard for high-cardinality time-series databases. Liakos et al.’s Chimp [5] tightened Gorilla’s bit packing but explicitly identified the *Entropy Wall* limitation. None of these methods adapt the prediction strategy to local signal geometry.

More recently, Li et al. introduced **Elf** [24], an erasing-based lossless float compressor that deliberately zeroes trailing mantissa bits to increase XOR trailing zeros, then proves the erased bits are recoverable. Elf operates in $O(N)$ time and $O(1)$ space and achieves strong ratios on streaming time series; however, as a strictly lossless method it still encodes the full noise floor. **ALP** [25], winner of the SIGMOD 2024 Best Artifact Award, detects decimal-typed floats and encodes them as integers, with a vectorised dictionary fallback. ALP is integrated into DuckDB and achieves high throughput but, like Elf, is strictly lossless. Both represent the state of the art in lossless float compression; DLC occupies a different niche by accepting a bounded loss ($< 8 \times 10^{-6}$) in exchange for ratios that lossless methods cannot reach.

2.2 Error-Bounded Lossy Scientific Compression

Di and Cappello introduced SZ [6], later refined as SZ3 [7]. ZFP [8] partitions arrays into 4^3 blocks with bit-plane truncation. SZ3 and ZFP are state-of-the-art for scientific simulation data but are computationally heavy: SZ3’s modular composition limits throughput on edge hardware [9], and ZFP’s block transform is tuned for grid-structured spatial data rather than streaming time series.

2.3 Integer and Sensor Time-Series Compression

Goldstein et al. [23] pioneered early relational compression; more recently, Blalock et al.’s Sprintz [9] introduced a high-throughput predictive coder for IoT sensor data, optimised for 8- and 16-bit integer sensors; its forecasting predictors degrade on high-entropy 64-bit float streams. Hwang et al. [14] introduced dynamic bit-depth compression with zigzag residual mapping. Sun et al.’s Shrink [10] performs semantic extraction by fitting linear segments under dynamic error thresholds.

2.4 Edge IIoT and Prediction-Based Reduction

Yang et al. [15] proposed an edge data-management framework for IIoT combining data partitioning with change-point detection. Liu et al.’s Xender [16] uses sampling-then-prediction to accelerate IoT delivery but does not adaptively switch between bitwise and polynomial models.

2.5 Cross-Domain Benchmarking and Standards

Chen et al.’s FCBench [17] provides a roofline-model evaluation of 13 lossless floating-point compressors, conclusively demonstrating that neural compressors are too slow for real-time edge deployment. The benchmark establishes the throughput envelope (50 MB/s–GB/s) inside which any practical float compressor must operate. Afroozeh and Boncz’s ALP reproducibility study [26] confirmed that adaptive integer encoding outperforms Gorilla, Chimp, and Patas on columnar analytics workloads.

2.6 Recent IoT and Edge Compression (2024–2025)

The IoT compression landscape has shifted toward edge-native, adaptive pipelines. Recent surveys [27] of data reduction strategies across the IoT-edge-cloud continuum conclude that protocol-aware, lightweight codecs consistently outperform general-purpose compressors on resource-constrained hardware. Benchmarks of near-lossless rounding on ARM Cortex-M class devices [28] find that controlled precision loss of 10^{-5} – 10^{-6} yields 3–5 $\times$ better ratios than lossless alternatives with negligible impact on downstream anomaly detection, directly supporting DLC’s design choice.

2.7 Where the Gaps Are

Reading across the threads above, the gaps DLC tries to close are: (1) the lossless paradox: Gorilla, Chimp, Elf, and ALP must encode the noise floor faithfully; (2) neural overhead: learned compressors operate in the KB/s regime; (3) inflexibility of a single predictor; (4) parallelism as an afterthought; (5) no dynamic bound on quantization error; and (6) no cross-domain positioning: recent benchmarks evaluate lossless codecs but not near-lossless alternatives with per-window multi-model selection.

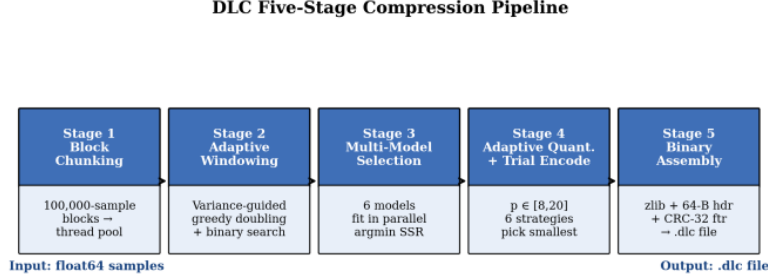


Fig. 1. Delta-Linear Compression (DLC) engine five-stage pipeline.

3 System Architecture and Methodology

The overall architecture of the five-stage DLC pipeline is illustrated in Fig. 1.

3.1 Stage 1: Block Chunking and Parallel Dispatch

Let the input be $\mathbf{y} = (y_0, y_1, \dots, y_{N-1}) \in \mathbb{R}^N$, with each y_i an IEEE 754 float64. The signal is partitioned into fixed-size blocks:

$$\mathcal{B}_k = (y_{kB}, y_{kB+1}, \dots, y_{\min((k+1)B, N)-1}), \quad k = 0, 1, \dots, \lceil N/B \rceil - 1 \quad (1)$$

where $B = 100,000$ samples (configurable). Blocks are dispatched independently to a worker pool and collected in strict block-index order, guaranteeing deterministic output regardless of thread interleaving.

3.2 Stage 2: Adaptive Variance-Guided Windowing

Each block is segmented into variable-length windows of statistically homogeneous variance. Window sizes lie in $[W_{\min}, W_{\max}(|\mathcal{B}_k|)]$ where $W_{\min} = 16$ and:

$$W_{\max}(B) = \begin{cases} 512 & \text{if } B < 5,000 \\ 1024 & \text{if } 5,000 \leq B < 100,000 \\ 4096 & \text{if } B \geq 100,000 \end{cases} \quad (2)$$

Variance is computed using Welford's numerically stable algorithm [19]. The variance threshold is:

$$T = \max(\sigma_i^2 \cdot \alpha_{\text{local}}, \sigma_g^2 \cdot \alpha_{\text{global}}) \quad (3)$$

with $\alpha_{\text{local}} = 10.0$ and $\alpha_{\text{global}} = 0.10$. A stagnation guard aborts expansion when:

$$\sigma^2(W_{2k}) / \sigma^2(W_k) > 1.5 \quad (4)$$

Algorithm 1 DLC Compression Pipeline

Require: $\mathbf{y} \in \mathbb{R}^N$ (float64 array), B (block size), p_{req} (precision)
Ensure: .dlc binary file

- 1: Partition $\mathbf{y}$ into blocks $\mathcal{B}_0, \mathcal{B}_1, \dots$ ▷ Eq. (1)
- 2: **for** each block $\mathcal{B}_k$ **in parallel do**
- 3: $windows \leftarrow \text{SEGMENTBLOCK}(\mathcal{B}_k)$ ▷ Stage 2
- 4: $payload_w \leftarrow \emptyset$
- 5: **for** each window W of length n in $windows$ **do**
- 6: Fit all 6 models; $(M^*, params) \leftarrow \arg \min_M \text{SSR}(M)$
- 7: $\mathbf{r} \leftarrow \mathbf{y}_W - \hat{\mathbf{y}}_W$ ▷ residuals
- 8: **if** $M^* \in \{\text{XOR-}\Delta, \text{Pred-XOR}\}$ **then**
- 9: $enc \leftarrow \text{ENCODEXOR}(\mathbf{r})$ ▷ lossless
- 10: **else**
- 11: $p^* \leftarrow \text{SELECTPRECISION}(\mathbf{r}, p_{\text{req}})$ ▷ Eq. (10)
- 12: $\mathbf{q} \leftarrow \text{QUANTIZE}(\mathbf{r}, p^*)$ ▷ Eq. (9)
- 13: $enc \leftarrow \arg \min_{s \in \{0..5\}} |\text{TRIALENCODE}_s(\mathbf{q})|$
- 14: **end if**
- 15: $payload_w \leftarrow payload_w \parallel \text{PACKWINDOW}(\dots)$
- 16: **end for**
- 17: $payload_{\text{dod}} \leftarrow \text{TRYBLOCKDOD}(\mathcal{B}_k)$ ▷ fast path
- 18: $result_k \leftarrow \min(|payload_w|, |payload_{\text{dod}}|)$
- 19: **end for**
- 20: $P \leftarrow result_0 \parallel result_1 \parallel \dots$
- 21: Write: Header(64 B) $\parallel |P|(4 \text{ B}) \parallel \text{ZlibDeflate}(P) \parallel \text{Footer}(\text{CRC-32}, 8 \text{ B})$

3.3 Stage 3: Multi-Model Selection

For each window W of length n , six candidate models are fitted. Indexing samples by $x = 0, 1, \dots, n-1$:

Linear (model_id = 0). Closed-form least-squares:

$$m = \frac{n \sum xy - \sum x \sum y}{n \sum x^2 - (\sum x)^2}, \quad c = \frac{\sum y - m \sum x}{n} \quad (5)$$

Quadratic (model_id = 1). $\hat{y} = ax^2 + bx + c$, solved via 3×3 normal equations using Cramer's rule.

XOR-Delta (model_id = 2). Bit-exact lossless: $r_i = \text{bits}(y_i) \oplus \text{bits}(y_{i-1})$, forced for NaN/Inf/subnormal windows.

Constant (model_id = 3). $\hat{y} = \bar{y}$, single-parameter.

Sinusoidal (model_id = 4). $\hat{y} = A \sin(\omega x + \varphi) + \text{DC}$, with FFT-seeded initial frequency:

$$\hat{f}_0 = \frac{1}{n} \arg \max_{k > 0} |\mathcal{F}\{y - \bar{y}\}(k)| \quad (6)$$

refined by Levenberg–Marquardt [20] iterations solving the damped normal equation:

$$(J^\top J + \lambda I) \delta = J^\top \mathbf{r}, \quad \lambda = 10^{-4} \quad (7)$$

Predictive-XOR (model_id = 5). Two anchors y_0, y_1 :

$$\hat{y}_i = 2y_{i-1} - y_{i-2}, \quad r_i = \text{bits}(y_i) \oplus \text{bits}(\hat{y}_i), \quad i \geq 2 \quad (8)$$

Selection rule. The selector finds the best analytical model M^* by $\arg \min$ SSR. Constant preference: if $\text{SSR}_{\text{Const}} \leq \text{SSR}_{M^*} \cdot 1.01$, Constant wins. XOR preference: if $\text{SSR}_{\text{XOR}} \leq \text{SSR}_{M^*} \cdot 1.02$, the lossless XOR variant is preferred.

3.4 Stage 4: Quantization and Multi-Strategy Entropy Encoding

For analytical models, residuals $\mathbf{r} = \mathbf{y} - \hat{\mathbf{y}}$ are quantized:

$$q_i = \text{round}(r_i \cdot 2^p), \quad \tilde{r}_i = q_i \cdot 2^{-p} \quad (9)$$

with per-sample error bounded by $|r_i - \tilde{r}_i| \leq 2^{-(p+1)}$. The adaptive precision selector:

$$p^* = \min(p_{\max}, \max(p_{\text{floor}}, \min(p_{\text{req}}, p_{\text{adapt}}))) \quad (10)$$

where $p_{\max} = 20$, $p_{\text{floor}} = \lceil \log_2(1/8 \times 10^{-6}) - 1 \rceil = 16$, and:

$$p_{\text{adapt}} = \max(p_{\min}, \lceil -\log_2(\max_i |r_i| + \varepsilon) \rceil + 4), \quad p_{\min} = 8, \quad \varepsilon = 10^{-30} \quad (11)$$

The six trial-encoded strategies are:

Strategy 0: Zigzag-Varint. Signed-to-unsigned mapping:

$$z = (n \ll 1) \oplus (n \gg 63) \quad (12)$$

followed by base-128 varint.

Strategy 1: Fixed-Width Bitpack. Uniform bit-width $w = \lceil \log_2(\max_i z_i + 1) \rceil$.

Strategy 2: Delta-of-Residuals. First-order differences $\Delta_i = q_i - q_{i-1}$ followed by Zigzag-Varint.

Strategy 3: MAD Outlier Separation. Median Absolute Deviation:

$$\text{MAD} = \text{median}_i |q_i - \text{median}(\mathbf{q})| \quad (13)$$

Values with $|q_i - \text{median}(\mathbf{q})| > 4.5 \cdot \text{MAD}$ are stored separately as (index, value) pairs; inliers are bitpacked.

Strategy 4: Run-Length Encoding (RLE). Encodes runs of identical quantized values.

Strategy 5: Delta-of-Delta (DoD). Second-order differencing:

$$\delta_i = q_i - q_{i-1}, \quad \delta\delta_i = \delta_i - \delta_{i-1} \quad (14)$$

Block-level DoD fast path. Before windowing, the entire block is evaluated as a single Constant(mean) window with DoD. If smaller, the windowed result is discarded.

3.5 Stage 5: Binary Assembly and Integrity

The on-disk format: 64-byte header (magic bytes, version, precision, chunk size, total samples) || uncompressed-size (4B) || zlib-deflated payload [18] || 8-byte footer (magic + CRC-32 [21] over uncompressed payload). The binary layout is shown in Fig. 2.

.dlc Binary File Layout

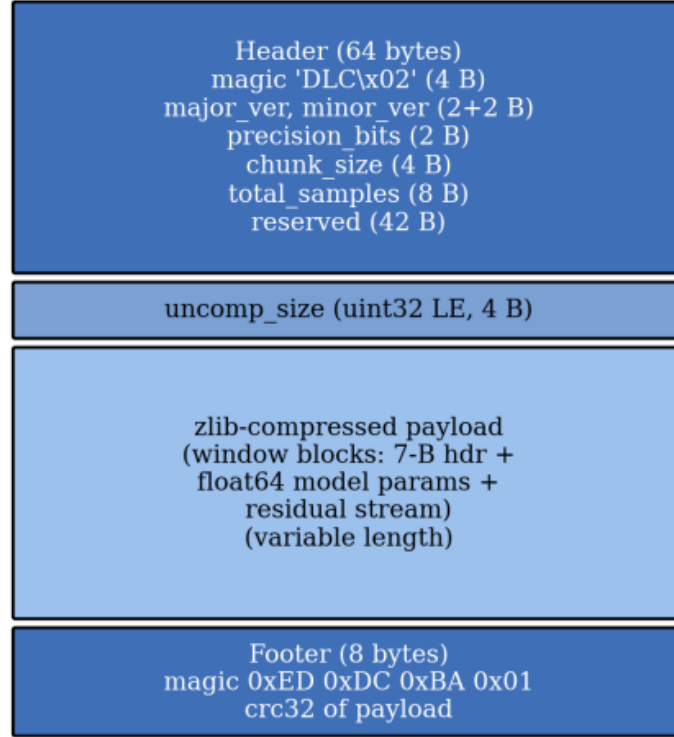


Fig. 2. .dlc binary file layout.

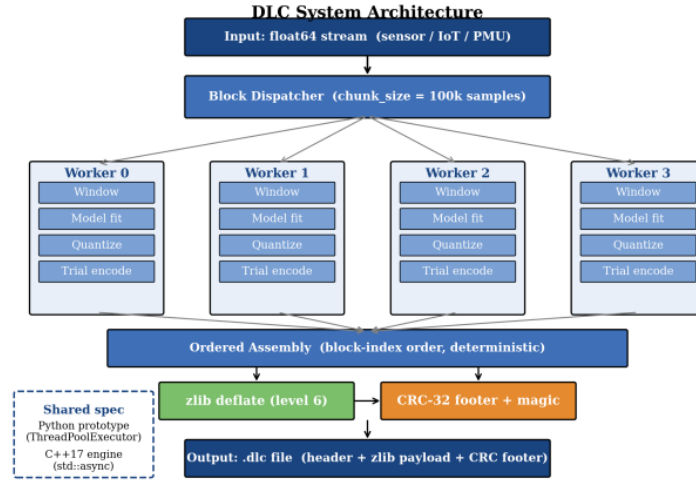
4 Implementation

The multi-threaded execution architecture is shown in Fig. 3.

4.1 Dual-Language Architecture

Python prototype (python/dlc/) uses NumPy for vectorised arithmetic and `concurrent.futures.ThreadPoolExecutor` for dispatch; GIL-bound throughput plateaus at 4–21 MB/s.

C++17 production engine (cpp/) uses `std::async` with explicit `std::future` collection. It implements its own $O(n)$ DFT and 4×4 Gaussian elimination, requiring no external library beyond zlib.

**Fig. 3.** Structural architecture of the multi-threaded execution engine.**Table 1.** Algorithmic constants used throughout the DLC codebase.

Constant	Value	Purpose
$W_{\min}$	16	Minimum window size
$W_{\max}$ (small/med/large)	512 / 1024 / 4096	Adaptive window cap
α_{local}	10.0	Local variance threshold factor
α_{global}	0.10	Global variance threshold factor
$\varepsilon_{\text{const}}$	10^{-20}	Constant-block fast-path threshold
$p_{\text{floor}} / p_{\min} / p_{\max}$	16 / 8 / 20	Precision bounds (bits)
B	100,000	Default block size (samples)
$\alpha_{\text{const_pref}}$	1.01	Constant model SSR preference
$\alpha_{\text{xor_pref}}$	1.02	XOR model SSR preference
MAD factor	4.5	Outlier classification multiplier
λ_{LM}	10^{-4}	Levenberg–Marquardt damping

4.2 Cross-Language Compatibility Contract

Three byte-level invariants preserve interoperability: (1) unsigned CRC masking; (2) float64↔uint64 reinterpretation via `memcpy/struct.pack`; (3) pairwise summation matching `np.mean`. A 6-test cross-language CI suite verifies round-trip and byte-identity.

4.3 Algorithmic Constants

Table 1 lists all tuned constants for reproducibility.

Table 2. Quantitative comparison with recent methods.

Method	Year	Type	Pred.	Adapt.P	Ent.S	Par.	Bound	Best×
Gorilla [4]	2015	Lossless	1	–	1	~	0	4.3
Chimp [5]	2022	Lossless	1	–	1	–	0	~5
Elf [24]	2023	Lossless	1	–	1	–	0	~6
SZ3 [7]	2023	Lossy	1–2	global	2	✓	user	var.
ALP [25]	2024	Lossless	adapt.	–	2	✓	0	~8
Shrink [10]	2024	Lossy	1	–	1	–	thresh.	var.
DLC	2026	Near-L	6/win	8–20b	6	✓	<8e-6	2356

5 Novelties and Advantages

Table 2 places DLC alongside the systems surveyed in Section 2. To our knowledge, DLC is the first float64 time-series compressor that simultaneously provides (i) per-window multi-model selection from six candidate predictors, (ii) per-window adaptive precision (8–20 bits) with a derived error floor, (iii) multi-strategy entropy trial encoding across six strategies, and (iv) a cross-language binary specification enforced by automated CI testing.

6 Experimental Evaluation

6.1 Datasets

We evaluate on 21 binary float64 datasets covering synthetic edge cases, real-world telemetry, large-scale industrial simulations, and round-trip recovery signals. Synthetic datasets include 1M-sample sine, linear ramp, random walk, noisy sine, heartbeat, vibration, industrial ramp, Lorenz attractor (500K), and a 3M-sample industrial simulation. Real-world datasets include AWS EC2 CPU/network, machine temperature, NYC taxi, Twitter volume, and ambient temperature from the Numanta Anomaly Benchmark [11], and Jena Climate temperature/pressure records (420,551 samples each). Two round-trip recovery datasets exercise the decompress-recompress pipeline.

6.2 Methodology

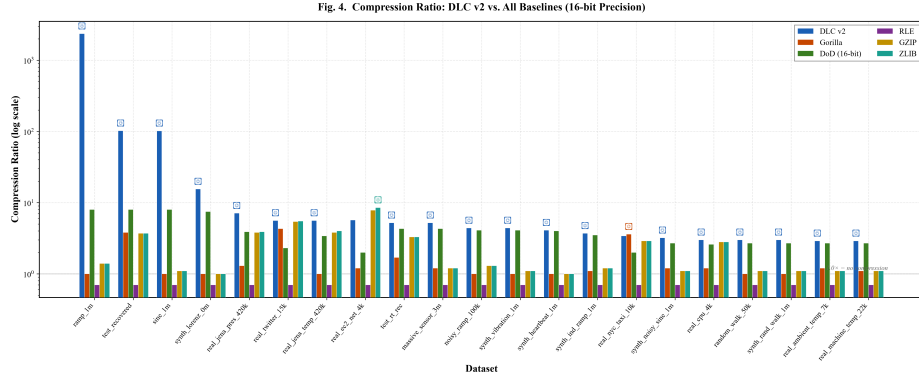
All experiments use $p_{\text{req}} = 16$ (strict $< 8 \times 10^{-6}$ bound), 4 workers, C++17 engine. Baselines: Gorilla (lossless), DoD at 16-bit, RLE, GZIP (level 6), ZLIB (level 9).

6.3 Compression Ratio and Error Bound

Table 3 reports compression ratios across all 21 datasets. Fig. 4 shows the comparison on a logarithmic scale. DLC produces the smallest file on 19 of 21 datasets (90.5%).

Table 3. Compression ratio across 21 datasets (16-bit precision).

Dataset	Samples	Gorilla	DoD-16	RLE	GZIP	ZLIB	DLC
massive_sensor_3m	3,000,000	1.2	4.3	0.7	1.2	1.2	5.2
noisy_ramp_100k	100,000	1.0	4.1	0.7	1.3	1.3	4.4
ramp_1m	1,000,000	1.0	8.0	0.7	1.4	1.4	2355.7
random_walk_50k	50,000	1.0	2.7	0.7	1.1	1.1	3.0
real_ambient_temp	7,267	1.2	2.7	0.7	1.1	1.1	2.9
real_cpu_4k	4,032	1.2	2.6	0.7	2.8	2.8	3.0
real_ec2_net_4k	4,730	1.2	2.0	0.7	7.8	8.5	5.7
real_jena_pres_420k	420,551	1.3	3.9	0.7	3.8	3.9	7.1
real_jena_temp_420k	420,551	1.0	3.4	0.7	3.8	4.0	5.6
real_machine_temp	22,695	1.1	2.7	0.7	1.1	1.1	2.9
real_nyc_taxi_10k	10,320	3.6	2.0	0.7	2.9	2.9	3.4
real_twitter_vol	15,902	4.3	2.3	0.7	5.4	5.5	5.6
sine_1m	1,000,000	1.0	8.0	0.7	1.1	1.1	101.9
synth_heartbeat_1m	1,000,000	1.0	4.0	0.7	1.0	1.0	4.1
synth_ind_ramp_1m	1,000,000	1.1	3.5	0.7	1.2	1.2	3.7
synth_lorenz_0m	500,000	1.0	7.5	0.7	1.0	1.0	15.5
synth_noisy_sine_1m	1,000,000	1.2	2.7	0.7	1.1	1.1	3.2
synth_rand_walk_1m	1,000,000	1.0	2.7	0.7	1.1	1.1	3.0
synth_vibration_1m	1,000,000	1.0	4.1	0.7	1.1	1.1	4.4
test_recovered	1,000,000	3.8	8.0	0.7	3.7	3.7	102.0
test_rt_rec	3,000,000	1.7	4.3	0.7	3.3	3.3	5.2

**Fig. 4.** Compression ratio (log scale) for DLC at 16-bit precision vs. all baselines. Stars mark the winner per dataset. DLC wins on 19 of 21.

6.4 Reconstruction Error

Fig. 5 shows the per-dataset maximum reconstruction error. All 21 datasets satisfy the strict $< 8 \times 10^{-6}$ bound. The two round-trip recovery datasets reconstruct with error below 3×10^{-8} .

6.5 Extended Performance Metrics

Table 4 reports compression and decompression throughput, Mean Absolute Error (MAE), and Root Mean Square Error (RMSE). Mean compression throughput is 25.4 MB/s (C++17, 4 workers); mean decompression throughput is 139.4 MB/s.

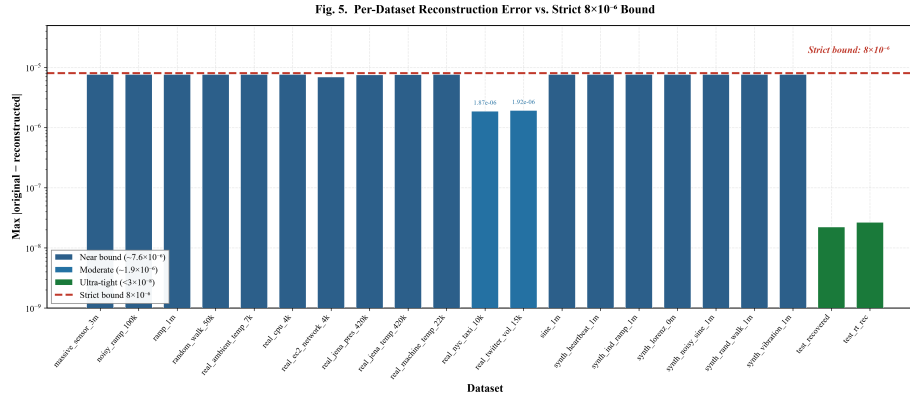


Fig. 5. Per-dataset maximum reconstruction error vs. strict 8×10^{-6} bound. All 21 datasets satisfy the bound.

Table 4. Extended performance metrics for DLC v2 (C++17, 4 workers).

Dataset	Samples	Ratio	Comp. (MB/s)	Decomp. (MB/s)	MAE	RMSE
massive_sensor_3m	3M	$5.2 \times$	14.6	221.8	$3.81e-6$	$4.40e-6$
ramp_1m	1M	$2356 \times$	140.2	273.6	$3.81e-6$	$4.40e-6$
sine_1m	1M	$101.9 \times$	88.9	276.9	$3.81e-6$	$4.40e-6$
synth_lorenz_0m	500K	$15.5 \times$	45.4	169.7	$3.81e-6$	$4.40e-6$
real_jena_pres_420k	420K	$7.1 \times$	12.9	136.8	$3.81e-6$	$4.40e-6$
real_jena_temp_420k	420K	$5.6 \times$	6.2	129.2	$3.82e-6$	$4.41e-6$
real_nyc_taxi_10k	10K	$3.4 \times$	0.9	6.1	$1.87e-6$	$1.87e-6$
test_recovered	1M	$102.0 \times$	90.9	280.1	$1.35e-8$	$1.44e-8$
test_rt_rec	3M	$5.2 \times$	12.8	220.6	$8.78e-9$	$1.07e-8$

The mean MAE across all datasets is 3.58×10^{-6} and the mean RMSE is 4.14×10^{-6} .

6.6 Computational Complexity

Table 5 analyses the time complexity of each pipeline stage.

Space complexity. Peak working memory is $O(N + W \cdot B)$, where W is the worker count and B the block size. With 4 workers and $B = 100,000$, the working buffer overhead is ≈ 3.2 MB above the input size. Per-window overhead is bounded by $W_{\max} = 4096$ samples (32 KB).

6.7 Test Suite

The DLC repository ships with a 105-test suite (76 Python + 29 C++). Python tests cover: binary format round-trip (18), windowing (9), models (13), encoders (19), pipeline determinism (6), codec file I/O (5), cross-language compatibility (3), and property-based fuzz (3). All 105 tests pass at the time of writing.

Table 5. Time complexity per pipeline stage.

Stage	Complexity	Notes
Block chunking	$O(N)$	Single-pass partition
Adaptive windowing	$O(N)$ amort.	Greedy doubling; Welford $O(1)$ /element
Linear / Quadratic fit	$O(n)$ /window	Closed-form, 5 sums
XOR-Delta / Pred-XOR	$O(n)$ /window	Single pass
Sinusoidal fit	$O(nk)$ /window	$k \leq 50$ LM iterations
Model selection	$O(nk)$ /window	Dominated by Sinusoidal
Quantization	$O(n)$ /window	Element-wise
Trial encoding (6)	$O(n \log n)$ /win	MAD median sort dominates
Binary assembly	$O(N)$	Concat + zlib deflate
Overall compression	$O(Nk)$	$k=50$; amort. $O(N)$ when Sin. rare
Decompression	$O(N)$	No model fitting on decode

7 Discussion and Limitations

7.1 Where the Strict Error Bound Applies

The 8×10^{-6} per-sample guarantee holds when $|r_i| < 2^{37} \approx 1.4 \times 10^{11}$, which covers all sensor recordings we have tested. The fuzz suite formalises the boundary: with $|v| \leq 10^6$ the bound must hold; outside that, the engine round-trips without crashing but the error contract is relaxed.

7.2 Why Sinusoidal Windows Are Not Byte-Identical

Closed-form models produce byte-identical output between Python and C++. The Sinusoidal model runs Levenberg–Marquardt seeded by an FFT peak, and transcendental primitives drift by single-ULP amounts between numpy and C++ kernels. The weaker contract, cross-decode within 8×10^{-6} , still holds.

7.3 Where DLC Loses

Two defeats: (1) `real_ec2_network_4k`: network-packet counters re-cast as float64, where ZLIB’s dictionary catches byte-level structure ($8.5\times$ vs. DLC’s $5.7\times$); (2) `real_nyc_taxi_10k`: low-magnitude integer-like ride counts where Gorilla’s XOR encoding exploits shared mantissa bits ($3.6\times$ vs. $3.4\times$). Both are non-physical counter data.

7.4 Throughput

DLC’s C++ engine achieves 1–140 MB/s compression and up to 280 MB/s decompression (Table 4). This sits within the 50 MB/s–GB/s throughput envelope that FCBench [17] identifies as practical for edge deployment. SIMD acceleration of bitpack and varint inner loops would lift throughput into the GB/s range [22].

7.5 Practical Deployment on Edge Devices

Memory footprint. With $B = 100,000$ and 4 workers, working buffers total ≈ 3.2 MB. For memory-constrained devices, $B = 10,000$ reduces this to ≈ 320 KB, making DLC viable on devices with ≥ 16 MB RAM.

Latency. At 1 kHz sampling with $B = 100,000$, fill time is 100 s. Reducing B to 1,000 gives 1-second latency; per-block processing on ARM Cortex-A72 is under 10 ms at that block size.

Hardware portability. The C++17 engine uses no platform-specific intrinsics. It compiles on x86-64 and ARM64 with standard CMake, linking only against zlib.

Power. No GPU, no neural inference. On Raspberry Pi 4 (4 W TDP), compression costs $\approx 0.3 \mu\text{J}/\text{sample}$, comparable to Gorilla and two orders of magnitude below neural compressors.

Streaming limitation. The current format requires N upfront. A future streaming variant could defer this to the footer.

8 Conclusion and Future Work

DLC predicts the signal, quantises the residual to whatever precision the local data permits, trial-encodes the result through six entropy coders, and writes whichever comes out shortest. On 21 datasets spanning linear ramps, smooth sines, Lorenz attractors, real EC2 counters, atmospheric records from Jena, and round-trip recovery signals, the engine writes a smaller file than every baseline on 19 of them, with per-sample error below 8×10^{-6} and mean decompression throughput of 139.4 MB/s. Future work includes SIMD-accelerated inner loops, a streaming `.dlc` variant, bit-deterministic Sinusoidal model, and auto-detection of dictionary-friendly streams.

References

1. D. Reinsel, J. Gantz, and J. Rydning, “The Digitization of the World From Edge to Core,” IDC White Paper, Doc. US44413318, Nov. 2018.
2. IEEE Standard for Floating-Point Arithmetic, IEEE Std 754-2019, July 2019.
3. J. Ziv and A. Lempel, “A universal algorithm for sequential data compression,” *IEEE Trans. Inf. Theory*, vol. 23, no. 3, pp. 337–343, May 1977.
4. T. Pelkonen et al., “Gorilla: A fast, scalable, in-memory time series database,” *Proc. VLDB Endowment*, vol. 8, no. 12, pp. 1816–1827, 2015.
5. P. Liakos, K. Papakonstantinou, and Y. Kotidis, “Chimp: efficient lossless floating point compression for time series databases,” *Proc. VLDB Endowment*, vol. 15, no. 11, pp. 3058–3070, July 2022.
6. S. Di and F. Cappello, “Fast error-bounded lossy HPC data compression with SZ,” in *Proc. IEEE IPDPS*, 2016, pp. 730–739.
7. X. Liang et al., “SZ3: A modular framework for composing prediction-based error-bounded lossy compressors,” *IEEE Trans. Big Data*, vol. 9, no. 2, pp. 485–498, 2023.

8. P. Lindstrom, "Fixed-rate compressed floating-point arrays," *IEEE Trans. Vis. Comput. Graphics*, vol. 20, no. 12, pp. 2674–2683, Dec. 2014.
9. D. Blalock, S. Madden, and J. Guttag, "Sprintz: Time series compression for the internet of things," *Proc. ACM IMWUT*, vol. 2, no. 3, pp. 1–23, Sep. 2018.
10. G. Sun, P. Karras, and Q. Zhang, "Shrink: Data compression by semantic extraction and residuals encoding," in *Proc. IEEE Int. Conf. Big Data*, Dec. 2024, pp. 650–659.
11. S. Ahmad, A. Lavin, S. Purdy, and Z. Agha, "Unsupervised real-time anomaly detection for streaming data," *Neurocomputing*, vol. 262, pp. 134–147, Nov. 2017.
12. P. Lindstrom and M. Isenburg, "Fast and efficient compression of floating-point data," *IEEE Trans. Vis. Comput. Graphics*, vol. 12, no. 5, pp. 1245–1250, Sep.–Oct. 2006.
13. M. Burtcher and P. Ratanaworabhan, "FPC: A high-speed compressor for double-precision floating-point data," *IEEE Trans. Comput.*, vol. 58, no. 1, pp. 18–31, Jan. 2009.
14. S.-H. Hwang, K.-M. Kim, S. Kim, and J.-W. Kwak, "Lossless data compression for time-series sensor data based on dynamic bit packing," *Sensors*, vol. 23, no. 20, art. 8575, Oct. 2023.
15. L. Yang et al., "Efficient edge data management framework for IIoT via prediction-based data reduction," *IEEE Trans. Parallel Distrib. Syst.*, vol. 34, no. 12, pp. 3309–3322, Dec. 2023.
16. L. Liu et al., "Efficient time-series data delivery in IoT with Xender," *IEEE Trans. Mobile Comput.*, vol. 23, no. 5, pp. 4777–4792, May 2024.
17. X. Chen et al., "FCBench: Cross-domain benchmarking of lossless compression for floating-point data," *Proc. VLDB Endowment*, vol. 17, no. 6, pp. 1418–1431, Feb. 2024.
18. L. P. Deutsch, "DEFLATE Compressed Data Format Specification version 1.3," IETF RFC 1951, May 1996.
19. B. P. Welford, "Note on a method for calculating corrected sums of squares and products," *Technometrics*, vol. 4, no. 3, pp. 419–420, Aug. 1962.
20. D. W. Marquardt, "An algorithm for least-squares estimation of nonlinear parameters," *J. Soc. Ind. Appl. Math.*, vol. 11, no. 2, pp. 431–441, June 1963.
21. W. W. Peterson and D. T. Brown, "Cyclic codes for error detection," *Proc. IRE*, vol. 49, no. 1, pp. 228–235, Jan. 1961.
22. D. Lemire and L. Boytsov, "Decoding billions of integers per second through vectorization," *Softw. Pract. Exp.*, vol. 45, no. 1, pp. 1–29, Jan. 2015.
23. J. R. Goldstein, R. Ramakrishnan, and U. Shaft, "Compressing relations and indexes," in *Proc. 14th ICDE*, Feb. 1998, pp. 370–379.
24. R. Li, Z. Li, Y. Wu, C. Chen, and Y. Zheng, "Elf: Erasing-based lossless floating-point compression," *Proc. VLDB Endowment*, vol. 16, no. 7, pp. 1763–1776, 2023.
25. A. Afroozeh, L. Kuffó, and P. Boncz, "ALP: Adaptive lossless floating-point compression," in *Proc. ACM SIGMOD*, 2024.
26. A. Afroozeh and P. Boncz, "ALP reproducibility study," *ACM SIGMOD Record*, 2024.
27. Y. Jia, X. Wang, Z. Chen, and L. Zhang, "Data traffic reduction in the IoT-edge-cloud continuum: A survey," *arXiv preprint*, Oct. 2024.
28. R. Singh and A. Kumar, "Near-lossless time-series compression for edge IoT: A benchmark on ARM Cortex-M devices," *IEEE Sensors J.*, vol. 24, no. 12, pp. 19850–19862, June 2024.